

TECHNICAL SOURCE AUDIT

Range-aware certificates for Asymptotic

Additional findings, exact-arithmetic countermodels,
and candidate implementations

Repository	VladimirReshetnikov/Asymptotic
Review date	September 10, 2026
Target evaluators	Wolfram Language 15 and Mathics3
Pinned revision	8cee870994f506b501bae3ea6bd4a3a7edb895c1

Principal result. The additional findings concern certificate completeness and wasted proof work: an odd-power interval enclosure loses sign-dependent information; rational powers inherit an avoidable exponential-magnitude restriction; and invariant failures can consume every arithmetic retry. None of these findings alleges that the inspected interval arithmetic produces an unsound successful certificate.

Evidence boundary. The repository was inspected through its GitHub sources. Exact-rational Python models and independent special-function controls were executed. No Wolfram or Mathics package execution succeeded in this environment. The supplied Wolfram Language programs are characterization probes and candidate code, not accepted package patches.

This article is a delta against the maintained review registers and the wave-5 review index. It does not reissue the existing backlog as new findings. The source-coverage and novelty boundaries are recorded explicitly.

Executive assessment

The package has several distinct computational layers: real asymptotic construction, generalized-series operations, specialized coordinate and special-function engines, native-result compatibility, and exact inverse certificates. A defect in one layer need not imply a defect in the others. In particular, a refusal by the exact certificate evaluator does not invalidate the finite asymptotic expression that supplied its initial approximation. The public interfaces and the certificate implementation make that distinction essential to this review.[1, 2]

After excluding the mechanisms already catalogued in the maintained review registers, three additional observations remain worth acting on. Their common feature is that arithmetic precision cannot recover information discarded by an inappropriate representation or repair a logically invariant input failure.

ID	Finding	Consequence	Evidence
N01	Odd powers of intervals crossing zero use dependent factors as independent intervals.	A polynomial whose derivative is at least $15/16$ on the verification interval receives derivative enclosure $[0, 5]$.	Source derivation; exact-rational model.
N02	Noninteger rational powers are enclosed through $\exp(p \log x)$.	A simple square-root problem with exact rational endpoints hits the exponential-magnitude cap although its algebraic enclosure is immediate.	Source derivation; exact-rational magnitude and root checks.
N03	The retry loop increases arithmetic order after failures that cannot depend on that order.	A definitely wrong source-side interval is checked seven times under the default refinement budget; larger budgets continue after the arithmetic cap.	Source control-flow derivation; independent state-machine model.

N01 and N02 are conservative coverage limitations, not false theorems. N03 is a performance and diagnostic-policy issue. The strongest immediate repair is N01: it changes one arithmetic primitive, has a short proof, preserves the existing real-domain restrictions, and admits an exact regression with small rational numbers. N02 needs an additional bounded root primitive. N03 needs a deliberately narrow distinction between an arithmetic-dependent failure and one that arithmetic refinement cannot alter.

The bundle contains 22 passing Python test methods, including 3,036 admissible generated integer-power interval cases, 7,800 integer-root cases, and 800 generated rational-root cases. A separate program passes 360 numerical controls for selected special-function formulas. These totals are not Wolfram or Mathics test totals, and the numerical controls are not certificates.

Contents

Executive assessment	1
1 Scope, revision, and exclusion method	3
1.1 What was inspected	3
1.2 How existing findings were excluded	3
1.3 Evidence terminology	3
2 Architecture relevant to the findings	4
2.1 Construction, representation, and verification are separate	4
2.2 Why the findings apply to both target evaluators	4
3 N01: odd interval powers discard useful dependence	4
3.1 The existing algorithm	4
3.2 A strict-monotonicity witness	5
3.3 The general pattern	6
3.4 An endpoint-range replacement	6
3.5 Proof of the candidate's enclosure property	6
3.6 Acceptance and performance interpretation	7
4 N02: algebraic powers inherit a transcendental resource limit	7
4.1 The source route	7
4.2 An exact square-root witness	7
4.3 A dyadic enclosure for a rational root	8
4.4 Choosing a significant-bit grid	8
4.5 Integer Newton iteration	9
4.6 From roots to rational powers	9
4.7 Resource policy and integration boundary	9
5 N03: arithmetic refinement repeats invariant failures	10
5.1 What changes after a failed attempt	10
5.2 A deterministic source-side witness	10
5.3 Why this is more than a cosmetic diagnostic	10
5.4 A deliberately narrow retryability contract	11
5.5 Interaction with N01 and N02	11
6 Additional mathematical source checks	12
6.1 Why include successful checks	12
6.2 Half-integer modified Bessel identities	12
6.3 Terminating confluent hypergeometric U	12
6.4 Incomplete-gamma Frobenius offsets	12
6.5 Lerch moment remainder bound	13
7 Compatibility and implementation recommendations specific to this delta	13
7.1 Keep the algebraic primitive independent of transcendental conversion	13
7.2 Keep candidate installation reversible	13
7.3 Preserve arithmetic-policy failures	14
7.4 Expose the reason for an unproductive retry	14
8 Validation plan and acceptance gates	14
8.1 Gate A: reproduce the unmodified behavior	14
8.2 Gate B: characterize the range-aware integer primitive	14
8.3 Gate C: admit the rational-power path	15
8.4 Gate D: restrict terminal retry classification	15
8.5 What the executed tests establish	15
9 Development sequence and conclusion	15
A Source-coverage record	16
B Bundle contents and use	16

1 Scope, revision, and exclusion method

1.1 What was inspected

The audit is pinned to 8cee870994f506b501bae3ea6bd4a3a7edb895c1. Findings apply to that source snapshot, not to an unobserved later version of the branch. The kernel inventory, public entry definitions, Mathics adapters, exact special-function identities, parameterized Frobenius dispatch, Dirichlet-series implementation, and inverse certificate engine were inspected. The certificate engine received the deepest line-by-line treatment. Appendix A states which files were read in full and which were sampled.

This is not an assertion that every kernel line, vendored article, test, build tool, and historical review payload was independently re-audited. The repository is large enough, and its historical review corpus extensive enough, that such a claim would be misleading. The deliverable is a substantial source survey followed by a deeper, non-duplicating treatment of the three mechanisms above.

1.2 How existing findings were excluded

The maintained CODE_REVIEW_STATUS.md consolidates waves 1 and 2 and links the complete later intakes. WAVE_3_INTAKE.md and WAVE_4_INTAKE.md give mechanism-level crosswalks. The wave-5 index describes the subsequently supplied reports. Retired reports remain part of the exclusion set through the retained crosswalks and tombstones.[3, 4, 5, 6]

The exclusion check used those maintained registers and the wave-5 index, rather than treating the lack of a search hit as proof of novelty. An attempted GitHub code search returned an incomplete result set; it was not used to establish absence. The review does not claim a fresh full-text comparison against every unnumbered paragraph of all historical articles.

The nearest older work to N01 and N02 is the broad certificate-extension item X05. That item already proposes stronger polynomial enclosures. The present contribution is not another recommendation to “use better interval arithmetic.” It identifies the specific odd-power factorization that loses a strictly positive derivative bound, supplies exact small witnesses, and develops an endpoint-based replacement. Likewise, N02 isolates an algebraic operation being charged to a transcendental magnitude budget and supplies a constructive rational root enclosure. Those concrete mechanisms are not identified in the inspected consolidated ledgers.

N03 is adjacent to the existing accuracy-planning work but concerns a different state transition: repeated evaluation after a failure whose premises are unchanged. This article does not repeat recommendations about relative-tolerance planning, numerical precision promotion, branch identification at remote roots, repeated affine classification, or general certificate subdivision.

1.3 Evidence terminology

Source-derived means that the stated algorithm or control flow follows from the inspected definitions. *Exact model* means that a separately written Python implementation was executed using integers and `Fraction`; it is not an execution of the repository. *Numerical control* means an independent high-precision comparison with a numerical oracle. *Public probe* means supplied Wolfram Language code whose result still needs to be recorded in the target evaluator.

No successful Wolfram package run was available: the Wolfram connector reported that it required reconfiguration. Mathics was not installed, and an installation attempt failed because package-network resolution was unavailable. This affects evaluator-specific validation, not the elementary interval counterexamples proved below. The candidate WL installation and restoration code also remains unexecuted.

2 Architecture relevant to the findings

2.1 Construction, representation, and verification are separate

The public `AsymptoticExpansion` interface uses an exclusive exponent cutoff in its local coordinate. Specialized families can instead carry an extracted prefactor or a different natural coordinate. A `GeneralizedSeries` retains more than its finite expression: assumptions, approach information, model data, and error-related fields can matter to subsequent operations. The native compatibility path and the package's analytic path do not have interchangeable evidence contracts.[1]

The certificate layer has a different task. It attempts to prove the existence and uniqueness of a real root of a stored explicit forward function in a supplied interval. Its seed may come from an asymptotic approximation, but its proof is based on rational interval arithmetic, continuity, derivative separation, and residual containment or endpoint signs. The certificate therefore has its own capability boundary.[2]

A useful way to interpret the findings is to follow the data through three stages:

explicit function and branch $\longrightarrow$ interval evaluation $\longrightarrow$ proof attempt and retry policy.

N01 loses useful information during interval evaluation. N02 selects an unnecessarily restrictive implementation of interval evaluation. N03 assumes that a failed proof attempt will benefit from more arithmetic even when the relevant inputs and predicates cannot change.

2.2 Why the findings apply to both target evaluators

The implicated certificate functions are shared package code. Their arithmetic is expressed in exact integers and rationals, not in a Wolfram-only numerical library. The Mathics compatibility layer changes evaluation machinery and selected primitives, but the inspected odd-power multiplication structure and certificate retry policy are not separate algorithms under the two engines.[2, 7, 8]

That establishes a shared source-level concern. It does not establish identical observed diagnostics, timings, or public construction success on both engines. A Mathics refusal earlier in construction could prevent a public probe from reaching the certificate primitive. A symbolic simplification in Wolfram could also change an expression before it reaches a particular route. For that reason, the supplied probes include both the private arithmetic call and a public constructor-to-certificate call.

3 N01: odd interval powers discard useful dependence

3.1 The existing algorithm

`certIntegerPower` uses binary exponentiation. It multiplies the accumulated answer by the current interval when a bit of the exponent is set. Its squaring step is already range-aware: when an interval crosses zero, it replaces the lower square bound by zero rather than using a generic independent interval product. This special squaring rule is sound and useful.[2]

The loss occurs when an odd power subsequently multiplies the original interval by an even power of the same variable. The two factors are then treated as independently varying. For $X = [-1/4, 1]$, the source algorithm computes

$$X^2 = [0, 1], \quad X \cdot X^2 = [-1/4, 1].$$

The actual range of x^3 on X is

$$\{x^3 : x \in X\} = [-1/64, 1].$$

The lower endpoint $-1/4$ in the computed enclosure combines $x = -1/4$ from one factor with $x^2 = 1$ from the other. No single $x \in X$ realizes that combination.

This is the familiar dependency phenomenon of interval arithmetic, but the implementation issue is unusually local: an integer-power primitive knows that the factors represent the same scalar. It need not forget that information.

3.2 A strict-monotonicity witness

Consider

$$f(x) = x + x^4, \quad I = [-1/4, 1], \quad f'(x) = 1 + 4x^3.$$

Since x^3 is increasing,

$$f'(I) = [15/16, 5].$$

Thus f is strictly increasing throughout I , with a strictly positive derivative margin. The source algorithm instead obtains

$$1 + 4[-1/4, 1] = [0, 5].$$

The certificate routine sets its derivative lower bound to zero whenever neither endpoint of the derivative enclosure establishes a strict sign. It consequently refuses this derivative enclosure with `DerivativeNotSeparated`.^[2]

There is no numerical ambiguity in the example. Every number involved in the offending calculation is dyadic, so the directed-rounding step leaves it unchanged at the supported arithmetic orders. Increasing `EnclosureOrder` cannot recover the lost correlation.

The root problem $f(x) = 0$ is equally simple: $x = 0$ is the unique root in I . Indeed,

$$f(-1/4) = -63/256 < 0, \quad f(1) = 2 > 0.$$

The exact center $c = 0$ has residual zero. With the correct positive derivative enclosure, the residual certificate can close immediately.

To tie the example to a selected local branch rather than an unrelated remote root, construct the inverse from $x_0 = -1/2$, approaching from above. On the entire ray $x > -1/2$,

$$f'(x) > 1 + 4(-1/2)^3 = 1/2.$$

The inverse branch issuing from $f(-1/2) = -7/16$ is therefore unambiguous on that ray and includes the verification interval. This is not the previously reported issue of identifying a distant root with the intended inverse branch.

```
s = AsymptoticInverse[x + x^4, {x, -1/2}, {y, 3},
  Direction -> "FromAbove"];
InverseCertificate[s, 0,
  "Interval" -> {-1/4, 1}, "Center" -> 0,
  "EnclosureOrder" -> 2,
  "RefineExpansion" -> False, "MaxRefinements" -> 0]
```

This public call is a supplied, unrun probe. The private arithmetic result and its mathematical obstruction were independently verified. The public probe must additionally confirm constructor admission and the exact retained source-domain predicate.

3.3 The general pattern

The effect is not restricted to the displayed cubic. Let $0 < a < b$ and $X = [-a, b]$. An independent product enclosure for $x x^{2k}$ can have lower endpoint $-ab^{2k}$, although the true lower endpoint of x^{2k+1} is $-a^{2k+1}$. Their absolute ratio is

$$\left(\frac{b}{a}\right)^{2k}.$$

Binary exponentiation does not produce exactly this expression for every exponent, but the formula explains why sign-asymmetric zero-crossing intervals are a particularly valuable test family. Increasing precision controls rounding error; it does not control this dependency loss.

The reflected interval $[-1, 1/4]$ gives the corresponding upper-endpoint loss. Symmetric intervals can conceal the issue, because their odd-power hull is already symmetric and the independent products can accidentally be sharp.

3.4 An endpoint-range replacement

Proposition 1. *For an integer $n > 0$ and an ordered real interval $[a, b]$, the exact range of x^n is*

$$\begin{cases} [a^n, b^n], & n \text{ odd}, \\ [a^n, b^n], & n \text{ even and } 0 \leq a, \\ [b^n, a^n], & n \text{ even and } b \leq 0, \\ [0, \max(a^n, b^n)], & n \text{ even and } a < 0 < b. \end{cases}$$

Proof. An odd power is increasing on the real line. An even power is decreasing on the negative half-line and increasing on the positive half-line, with minimum zero at the origin. These monotonicity statements give the endpoints directly. $\square$

There is no need to form enormous exact endpoint powers before rounding. The existing bounded-exponent binary routine can be retained as a *point-power enclosure* routine. Run it on $[a, a]$ and $[b, b]$, then select the outward lower and upper endpoints according to the proposition. For negative exponents, first apply the existing reciprocal interval operation, retaining its rejection when zero lies in the base interval, and then apply the positive-exponent rule.

The supplied candidate uses precisely this arrangement. It preserves the existing exponent limit of 100,000 and the existing directed-dyadic multiplication policy. These are not a new universal memory guarantee: exact rational storage still depends on the magnitude of the represented values. The improvement is a tighter range abstraction, not an assertion that exponentiation has become constant-space.

3.5 Proof of the candidate's enclosure property

Suppose the retained point-power worker gives enclosures $A \ni a^n$ and $B \ni b^n$. For odd n , $[\inf A, \sup B]$ encloses every x^n with $a \leq x \leq b$. The three even-power cases follow by reversing endpoint roles or inserting zero. The proof requires only sound point enclosures; they need not be exact singletons.

For $n < 0$, zero exclusion ensures that the reciprocal map is continuous on the interval, and the existing reciprocal enclosure contains its image. Applying the positive-power construction to that image gives a sound enclosure for x^n . The zero-exponent case retains the source primitive's existing convention and is not used to expand the admissible domain of a symbolic zeroth-power expression.

3.6 Acceptance and performance interpretation

The independent tests check the small witness exactly, its reflection, zero and negative exponents, one-sided negative intervals, exact point intervals, and generated rational intervals. The reference oracle uses direct rational endpoint exponentiation and the mathematical range cases, rather than the candidate's rounded binary worker.

For 3,036 admissible generated intervals selected from 4,000 deterministic candidates, the candidate enclosure contains the exact range and is contained in the source-model enclosure. Invalid negative powers of intervals containing zero are excluded from that generated admissible set and tested separately as refusals.

Two point evaluations can cost more than a single generic interval exponentiation. The immediate justification is proof completeness, not a claimed blanket speedup. Measure successful end-to-end certification, including avoided retries, separately from primitive throughput. Symmetric, one-sided, and already point-like intervals are essential unchanged controls.

4 N02: algebraic powers inherit a transcendental resource limit

4.1 The source route

After integer powers and exponentials have been dispatched, the general real-power branch in `certEnclose` encloses a positive-base expression through

$$u^p = \exp(p \log u).$$

That identity is valid on the admitted positive real domain. However, the interval exponential rejects a rational argument whose absolute value exceeds `ExponentMagnitudeLimit`, which defaults to 10,000.[2]

For irrational exponents this route is natural. For a rational exponent, especially $p = 1/2$, it can charge a purely algebraic operation to an avoidable transcendental budget. A resource refusal can then reflect the chosen proof implementation rather than a difficult root problem.

4.2 An exact square-root witness

Take

$$I = [2^{39998}, 2^{40002}], \quad f(x) = \sqrt{x}, \quad c = 2^{40000}, \quad y = 2^{20000}.$$

The exact function range is

$$f(I) = [2^{19999}, 2^{20001}],$$

and

$$f'(I) = [2^{-20002}, 2^{-20000}].$$

Every displayed endpoint is rational; the function value at the center equals the target exactly. The root is unique on the positive ray. The input integers have only tens of thousands of bits, not an intrinsically prohibitive combinatorial representation.

The exponential argument at even the lower endpoint is

$$\frac{1}{2} \log(2^{39998}) = 19999 \log 2 > \frac{39998}{3} > 10000,$$

using $\log 2 > 2/3$. The latter inequality follows from

$$\log 2 = 2 \operatorname{artanh}(1/3) = 2 \sum_{k \geq 0} \frac{(1/3)^{2k+1}}{2k+1} > 2/3.$$

The independent source-model logarithm calculation also checks that the actual directed lower enclosure exceeds 10,000 at orders 2, 4, 10, and 60. Thus the example is not merely comparing an exact logarithm with a possibly much lower computed bound.

```
s = AsymptoticInverse[Sqrt[x], {x, Infinity}, {y, 3}];
InverseCertificate[s, 2^200000,
  "Interval" -> {2^39998, 2^400002},
  "Center" -> 2^40000, "EnclosureOrder" -> 2,
  "RefineExpansion" -> False, "MaxRefinements" -> 0]
```

Again, the public call is unrun. The source-derived obstruction is isolated by the supplied private probe of `certEnclose[Sqrt[x], x, I, ctx]`. The separate exact candidate returns the displayed square-root and derivative intervals without evaluating an exponential.

Raising `ExponentMagnitudeLimit` is a possible workaround, not the proposed fix. It increases work in a transcendental evaluator for a problem that requires only integer comparisons. A domain-preserving rational-power branch is the more direct solution.

4.3 A dyadic enclosure for a rational root

Let $q = a/b > 0$ be rational and $d \geq 1$ be an integer. Choose a positive dyadic grid spacing $g = 2^s$. Define

$$T = \left\lfloor \frac{q}{g^d} \right\rfloor, \quad m = \lfloor T^{1/d} \rfloor.$$

Then

$$mg \leq q^{1/d} < (m+1)g.$$

The upper endpoint can be collapsed to the lower endpoint when $(mg)^d = q$ exactly.

Lemma 1. For $z \geq 0$ and integer $d \geq 1$,

$$\left\lfloor \lfloor z \rfloor^{1/d} \right\rfloor = \lfloor z^{1/d} \rfloor.$$

Proof. For any nonnegative integer m , the integer m^d satisfies $m^d \leq z$ if and only if $m^d \leq \lfloor z \rfloor$. The two expressions therefore choose the same largest integer m . $\square$

This reduces root enclosure to exact division and integer root extraction. It does not require a floating-point guess, a numerical sign test, or a general algebraic-number engine.

4.4 Choosing a significant-bit grid

For a requested working size $B \geq 2$, compute

$$e = \lfloor \log_2 q \rfloor, \quad k = \left\lfloor \frac{e}{d} \right\rfloor, \quad g = 2^{k-B+1}.$$

The integer e is obtained from numerator and denominator bit lengths plus one exact comparison; no transcendental logarithm is evaluated.

Write $e = kd + r$ with $0 \leq r < d$. Since $2^e \leq q < 2^{e+1}$,

$$2^k \leq q^{1/d} < 2^{k+1}.$$

Hence a noncollapsed enclosure of width g satisfies

$$\frac{\text{width}}{q^{1/d}} \leq 2^{1-B}.$$

Negative binary exponents require floor division, not truncation toward zero. Both implementations explicitly account for that point. The tests include small positive rational inputs across negative and positive scale exponents.

4.5 Integer Newton iteration

The reference implementation computes $\lfloor n^{1/d} \rfloor$ with integer Newton iteration. Starting from a power-of-two upper bound x , it forms

$$y = \left\lfloor \frac{(d-1)x + \lfloor n/x^{d-1} \rfloor}{d} \right\rfloor.$$

It replaces x by y while $y < x$, stopping otherwise. The initial exponent is chosen from the bit length of n . Special cases handle zero, one, degree one, and degrees at least the input bit length.

The real Newton step stays above the real root; taking the floor cannot move below its integer floor. While an integer iterate is larger than the integer root, the next integer step is strictly smaller. Thus termination identifies the integer root floor. The independent tests check the defining inequalities

$$m^d \leq n < (m+1)^d$$

for 7,800 small integer/degree pairs and additional large cases immediately below, at, and above exact powers.

The Python implementation also recognizes exact powers of two directly. The supplied WL candidate uses the general integer iteration; this is a deliberate implementation difference, not a claim that their performance is identical.

4.6 From roots to rational powers

For a reduced exponent p/d , first enclose the nonnegative d -th root of each base endpoint, then apply the sound integer-power primitive with exponent p . Negative p requires a strictly positive lower endpoint. Positive p may admit a zero lower endpoint. A degree-one exponent can use the existing integer route directly.

This does *not* adopt the real odd root of a negative number as the meaning of Wolfram Power. Noninteger Power uses its principal complex value; replacing $(-8)^{1/3}$ by -2 would change that contract. The candidate therefore admits only nonnegative bases for noninteger rational powers, with the stricter positivity condition for negative powers.[12]

The candidate first tests whether numerator and denominator are exact d -th powers. This returns exact rational roots such as $\sqrt[4]{9} = 2/3$ without forcing them onto a dyadic grid. For nonperfect roots, all bounds remain rational and are checked by powering the endpoints to d .

4.7 Resource policy and integration boundary

The reference candidate limits the integer exponent magnitude, root degree, input rational bit size, and the size of scaled numerator/denominator operands. Powers of two are cancelled before shifted operands are allocated. These checks are implementation bounds for the new primitive; they do not establish a strict wall-clock limit or bound every intermediate to exactly the same number of bits.

The default candidate limits are intentionally explicit: degree 128, integer exponent magnitude 100,000, and input/scaled operand size 1,000,000 bits. They are candidate values, not new proposed public defaults for the entire package. Benchmarking in both engines should determine an appropriate admission policy.

The WL file exposes `NonnegativeRationalPowerInterval` but does not automatically intercept `certEnclose`. A production integration should place a rational-exponent case before the general logarithm/exponential branch. Budget-limited inapplicability can retain the old route as a fallback; an admitted successful algebraic enclosure must not be weakened merely to reproduce the old magnitude refusal. Error tags and domain checks should remain deliberate rather than being changed accidentally by the new dispatch.

5 N03: arithmetic refinement repeats invariant failures

5.1 What changes after a failed attempt

The outer certificate loop appends each attempt to its history. Except for a special definitive-no-root condition, a failed attempt is normally retried until `MaxRefinements` is exhausted. Its arithmetic order is doubled up to 2,000. Interval contraction and recentering occur only after a successful certificate, not after a general failure.[2]

Consequently, on a persistent failure the function, target, interval, center, source side, and retained domain remain unchanged. Only the arithmetic order changes. This can be useful when directed rounding is the obstacle. It cannot be useful when an exact rational comparison already disproves the selected source-side condition.

5.2 A deterministic source-side witness

Construct an inverse at source point zero from above, for example from $f(x) = x + x^2$. Ask for a certificate with the interval $[-2, -1]$ and center $-3/2$. The interval is intentionally invalid for the selected source side. The correct behavior is a refusal.

```
s = AsymptoticInverse[x + x^2, {x, 0}, {y, 3},
  Direction -> "FromAbove"];
InverseCertificate[s, 1/10,
  "Interval" -> {-2, -1}, "Center" -> -3/2,
  "MaxRefinements" -> 6, "RefineExpansion" -> False]
```

The source-side check compares exact rational endpoints. Its answer does not depend on exponential/logarithm series order or dyadic precision. Nevertheless, the inspected persistent-failure path attempts orders

60, 120, 240, 480, 960, 1920, 2000

under the ordinary default starting heuristic. All seven attempts have the same mathematical obstacle.

The displayed sequence was checked in an independent control-flow model, not observed from a native run. The source loop and its update conditions establish the sequence. The public probe is supplied so that engine-specific diagnostics and history fields can be recorded.

5.3 Why this is more than a cosmetic diagnostic

The default small rational witness is inexpensive. Its importance is the retry policy it exposes. `MaxRefinements` is validated as a nonnegative integer without an upper bound in this function. A

large value can keep appending equivalent failure records after the arithmetic order has already reached its cap. More expensive structurally unsupported expressions can repeat substantial preceding work as well.[2]

No elapsed-time speedup is claimed here. The measurable invariant is the number of attempts that can possibly affect the outcome. For a proven arithmetic-independent refusal, the useful count is one, not seven or an arbitrarily large caller-selected number.

The resulting diagnostic also matters. A terminal source-side failure should not be framed primarily as exhaustion of an accuracy-refinement budget. The caller needs to change the interval or branch, not request more precision. The existing failure tag can be preserved while the stopping reason becomes more precise.

5.4 A deliberately narrow retryability contract

Add failure metadata such as

```
"ArithmeticRetryable" -> False
```

only where the implementation has a local proof that changing arithmetic order cannot help. Two initial producers are sufficient for a useful repair:

1. The first source-side failure in `certAttempt`, when the expansion point is an exact rational or one of the two real infinities. These comparisons are exact and the interval is unchanged after failure.
2. The final `UnsupportedEnclosure` branch of `certEnclose`, which rejects an expression head outside the evaluator's grammar. Increasing series order cannot add a new supported operation.

The driver stops immediately when this flag is explicitly false, retaining the original failure tag, attempt history, and any prior best certificate. Missing or unknown retryability preserves the current behavior.

Do not mark every `OutsideBranch`, `IntervalDomain`, or `DerivativeNotSeparated` failure as terminal. Some such failures result from over-enclosures that genuinely improve with precision. An unsupported predicate caught and converted to a generic false proof result also needs separate treatment; the proposed local flag must not silently imply that every generic domain refusal has been classified.

5.5 Interaction with N01 and N02

The explicit dyadic witness in N01 cannot improve with arithmetic precision. Other derivative-separation failures may improve. Fixing the range primitive is preferable to hard-coding a retry exception for that particular polynomial.

Similarly, N02 should be repaired by selecting algebraic enclosure when its hypotheses hold. A general exponential-magnitude refusal can depend on a rounded argument near the cap, so a blanket rule making that failure nonretryable would require more analysis.

These distinctions prevent the optimization from becoming a new unsound proof shortcut. The driver is not declaring that a root does not exist. It is declaring that *this arithmetic-only retry plan* cannot alter a particular already-established obstruction.

6 Additional mathematical source checks

6.1 Why include successful checks

A source review should not manufacture defects from every unfamiliar formula. Several inspected transformations have delicate signs, offsets, or subdominant terms. This section records independent checks that did *not* produce new findings, without presenting existing review recommendations as new work.

These checks cover mathematical formulas transcribed from the source. They do not test dispatch, assumption handling, serialization, coefficient admission, or runtime-specific evaluation of those formulas.

6.2 Half-integer modified Bessel identities

The identity module retains both exponentials in half-integer I_ν , rather than identifying the dominant large-argument expansion with the exact function. Starting from the half-integer hyperbolic formulas, its recurrence constructs higher orders; negative orders add the corresponding K_ν contribution.[9, 13]

The independent program checks nine half-integer magnitudes at four positive arguments, comparing positive-order I , negative-order I , and K with `mpmath`. All 108 controls pass at the stated tolerance. The tests deliberately include small arguments, where cancellation in the exponential representation is more severe. This is evidence for the transcribed identities on those inputs, not a stability guarantee for arbitrary order and argument.

6.3 Terminating confluent hypergeometric U

For a nonnegative integer n , the inspected finite polynomial is

$$U(-n, b, z) = \sum_{k=0}^n (-1)^{n+k} \binom{n}{k} (b+k)_{n-k} z^k.$$

It is consistent with the Laguerre-polynomial identity used by the source.[9, 14]

The independent controls test $n = 0, \dots, 8$, three nonintegral parameter values, and three positive arguments against `mpmath`'s U . All 81 cases pass. These tests do not expand admission beyond the source's resource bound or settle singular parameter conventions for other hypergeometric heads.

6.4 Incomplete-gamma Frobenius offsets

For positive a and positive z , the lower incomplete gamma contribution can be written

$$G_a(z) = \frac{z^a}{a} {}_1F_1(a; a+1; -z).$$

The two-argument upper gamma is $\Gamma(a) - G_a(z)$. A three-argument integral with a varying lower endpoint is a fixed contribution minus $G_a(z)$; with a varying upper endpoint, the sign reverses. The parameterized adapter's multiplier and offset logic agrees with these orientations.[10, 15]

Twenty-seven independent controls check the upper-gamma expression and both endpoint orientations for three positive parameters and three positive varying arguments. All pass. No sign or offset defect was established in this inspected path.

6.5 Lerch moment remainder bound

For fixed real s , $|z| < 1$, and $a > 0$, the defining sum is

$$\Phi(z, s, a) = a^{-s} \sum_{n \geq 0} z^n (1 + n/a)^{-s}.$$

The source expands $(1 + t)^{-s}$ and sums the Taylor coefficients against the geometric weights.[11, 16]

The remainder reasoning can be checked directly. Taylor's theorem gives a factor $(1 + \xi)^{-s-N}$ for $0 \leq \xi \leq t$. With

$$D = \max(0, \lceil -s - N \rceil),$$

its magnitude is at most $(1 + t)^D$. For $a \geq 1$, $t = n/a \leq n$, giving

$$|R_N| \leq a^{-s-N} \frac{|(s)_N|}{N!} \sum_{j=0}^D \binom{D}{j} M_{N+j}(|z|),$$

where $M_0(r) = 1/(1 - r)$ and $M_k(r) = \sum_{n \geq 0} n^k r^n$ for $k > 0$. This reproduces the source constant, including the otherwise easy-to-miss $n = 0$ contribution when $N = 0$.

The separate numerical program checks 144 combinations against a directly summed convergent defining series, including negative z , negative noninteger s , and terminating negative integer s . All pass with a numerical allowance far smaller than the nonzero bounds. Zero bounds in terminating cases are compared with a numerical tolerance, not asserted as exact numerical equalities. The displayed derivation, rather than those numerical comparisons, explains the mathematical form of the bound.

7 Compatibility and implementation recommendations specific to this delta

7.1 Keep the algebraic primitive independent of transcendental conversion

N02 offers a particularly useful shared implementation boundary for Wolfram and Mathics: integer arithmetic, exact division, ordered rational endpoints, and a root-floor certificate. Neither SymPy conversion nor arbitrary-precision numerical root finding is needed. This reduces the number of evaluator-specific behaviors on the proof path without changing the mathematical domain.

The test oracle should remain the defining integer inequalities, not equality of printed decimal approximations between engines. For a returned root interval $[l, u]$, checking $l^d \leq q \leq u^d$ is an exact acceptance criterion. For the composed rational power, use corresponding endpoint inequalities and preserve the sign restrictions.

7.2 Keep candidate installation reversible

CandidateCertificatePowers.wl installs nothing on load. Its explicit integer-power installer captures the original private definition, substitutes the range-aware wrapper, and exposes a restoration operation. It does not alter System'Power or other global system definitions. The rational-power primitive is exposed separately and is not silently inserted into the expression evaluator.

This is experimental scaffolding, not a preferred final package architecture. After validation, the production repair should be a source change in the certificate module, and generated distribution artifacts should be rebuilt through the repository's normal process. The candidate's reversible overlay is useful for differential probes in a fresh kernel; it is not a claim of safe migration across arbitrary package reloads or unrelated

source revisions. Evaluate individual probe expressions after installation: loading the complete probe script again reloads the original package and invalidates an overlay comparison.

7.3 Preserve arithmetic-policy failures

A tighter interval should not accidentally bypass the existing exponent check, zero-exclusion rule for reciprocals, or caller-request validation. The N01 worker delegates point computations to the captured original routine specifically to retain those policies.

The N02 primitive has additional admission limits. Its integration needs a distinct decision between inapplicability, budget-limited fallback, and a genuine domain violation. Do not return a guessed endpoint or a floating approximation merely because the exact root budget is exhausted. Likewise, do not label an exact algebraic success as a transcendental approximation.

7.4 Expose the reason for an unproductive retry

N03's proposed flag answers a narrow question: can the currently implemented arithmetic-order increase affect the failed predicate? It does not replace the failure tag and does not assert global impossibility of certification.

The diagnostic should leave the user with the right next action. A definitely wrong source-side interval needs a different interval or branch. An unsupported enclosure head needs a supported proof transformation or additional primitive. A genuinely precision-limited residual can continue through the existing adaptive arithmetic policy. Those cases should not all end with the same apparent recommendation to spend more refinement budget.

8 Validation plan and acceptance gates

8.1 Gate A: reproduce the unmodified behavior

Run `ProbeCertificates.wl` in separate fresh Wolfram 15 and Mathics3 processes. Record the actual kernel version, package revision, complete probe outcome, and any earlier constructor refusal. The private N01 and N02 probes isolate the arithmetic mechanism; the public probes establish reachability through actual result objects.

A constructor failure must be retained as such. It must not be rewritten in the report as a successful reproduction of the later certificate failure. Conversely, a private result that differs from the source model should trigger investigation of evaluator semantics before the candidate is integrated.

8.2 Gate B: characterize the range-aware integer primitive

After explicit candidate installation, rerun the private and public N01 probes. The target private results are $[-1/64, 1]$ and $[15/16, 5]$. Then cover odd and even exponents, both signs of one-sided intervals, symmetric and asymmetric zero-crossing intervals, point intervals, non-dyadic endpoints, negative powers, zero exclusion, and exponent-limit refusal.

Compare the same tests after restoration. Restoration is itself a test obligation because the WL installer was not exercised here. For accepted source integration, repeat the relevant tests using the modular and distributed package forms rather than relying only on an in-memory overlay.

8.3 Gate C: admit the rational-power path

Test exact rational roots, nonperfect roots, negative binary scales, zero bases with positive powers, reciprocal powers away from zero, and explicit refusals for negative bases. Check degree and scaled-operand limits before constructing large temporary values.

The large square-root witness should return its exact power-of-two endpoints through the candidate primitive. A full public certificate must still prove the retained source-domain condition and derivative separation. The primitive's success alone is not a substitute for those checks.

8.4 Gate D: restrict terminal retry classification

For the wrong-side rational interval, the target is one attempt with the original branch failure and a nonrefinable stopping reason. At least one ordinary finite-precision case that improves after raising arithmetic order must retain its retries. Unknown classifications should behave exactly as before.

Test `MaxRefinements` $\rightarrow 0$, the default budget, and a larger budget. Verify that a prior best certificate is retained if a later classified failure terminates the loop. Do not infer this property from the independent retry-order model: it must be checked against the actual association fields in the driver.

8.5 What the executed tests establish

The exact Python suite passed all 22 methods. The generated integer interval cases establish enclosure and containment relationships for the sampled exact inputs. The root-floor tests establish the integer inequalities on their finite test domain; the proof in Section 4 establishes why the algorithm works generally. The root-enclosure tests similarly check exact powered endpoint inequalities without a floating-point oracle.

The special-function program passed 360 numerical controls at 120 decimal working digits. Its largest scaled identity discrepancy was approximately 6.56×10^{-111} . That small discrepancy is a numerical observation from the independent implementation, not an error bound for the package or a new theorem about all parameters.

No package MUnit totals, portable-suite totals, native timings, or Mathics timings are reported as results of this review.

9 Development sequence and conclusion

Implement and validate N01 first. Its mathematical contract is small, its witness uses tiny exact numbers, and its remedy does not require a new expression grammar. Accept it on enclosure correctness and improved certificate reachability, not on an assumed universal speedup.

Develop N02 as a separately tested algebraic interval primitive. Its proof obligation is exact and portable. Keep the admission limits explicit, then integrate it before the general real-power route only after both engines pass the endpoint inequalities and refusal controls.

Implement N03 with local, explicit nonretryability evidence. Starting with a small set of indisputable producers is better than classifying whole families of failure tags. Preserve the current behavior whenever retryability is unknown.

The broader source survey did not justify inventing additional coefficient or identity defects. The useful delta is narrower and more concrete: retain the correlation that an integer-power node already knows, avoid transcendental work for admitted rational powers, and do not spend arithmetic refinement on a

predicate that cannot change. Each improvement has an independent mathematical oracle and a sharply defined integration boundary.

A Source-coverage record

The following scope describes this review's source reading, not runtime coverage. All repository references use the pinned revision.

Source	Treatment
src/Kernel/InverseCertificates.wl	Full focused reading; arithmetic and driver analysis.
src/Kernel/AsymptoticAnalysis.wl	Opening public definitions and request/assumption infrastructure sampled through the first 210 source lines.
MathicsCompatibility.wl, MathicsAssumptions.wl, MathicsAlgebra.wl MathicsNumerical.wl, MathicsTaylor.wl	Full adapter reading; no additional issue retained after exclusion. Full reading of numerical and defining-series adapters.
MathicsCalls.wl, MathicsCalculus.wl, MathicsInputAssumptions.wl	Full reading of held extraction, finite calculus, and membership-protection paths.
MathicsLists.wl, MathicsCoreFunctions.wl	Full reading of late Map and ProductLog-related adapters.
SpecialFunctionIdentities.wl	Full reading; selected independent formula controls.
ParameterizedSpecialFunctions.wl	Full reading; Frobenius orientation checks.
DirichletSpecialFunctions.wl	Full reading; Taylor/moment bound derivation and controls.
Kernel directory inventory	Module names and sizes surveyed; not every module read in full.
Maintained review registers and wave-5 index	Deduplication baseline, including retired-report crosswalks; not a fresh full-text audit of every historical article.

Paths without a directory prefix in this table are under `src/Kernel/`. Build tools, all tests, and the vendored research corpus are not claimed as fully inspected. The absence of a finding in this article is not a certificate of correctness for an unexamined subsystem.

B Bundle contents and use

Path	Purpose
article/article.tex, article/article.pdf	This article and editable source.

Path	Purpose
code/certificate_power_model.py	Executed exact-rational source model and candidate algorithms.
code/test_certificate_power_model.py	22 executed unittest methods with independent exact oracles.
code/check_special_identities.py	Executed independent numerical formula controls.
code/CandidateCertificatePowers.wl	Unrun reversible integer-power candidate and separate rational-power primitive.
code/ProbeCertificates.wl	Unrun private/public characterization probes.
patches/retry_policy.fragment.wl	Unrun source-edit fragments for narrow terminal-failure metadata.
evidence/python-tests.txt	Actual unittest output.
evidence/summary.json	Scope, exact model results, and test counts.
evidence/special-function-controls.json	Actual numerical-control summary.
evidence/novelty-and-scope.md	Mechanism-level exclusion and evidence boundaries.
README.md	Commands, dependencies, and integration cautions.

From the bundle's code directory, run the exact suite with

```
python -m unittest -v test_certificate_power_model
```

The exact suite needs only the Python standard library. The optional numerical controls need mpmath:

```
python check_special_identities.py
```

Compile the article from its directory using two passes of `pdflatex`. No checksum files are included. The commit identifier is provenance for the reviewed source, not a bundle checksum.

References

- [1] Vladimir Reshetnikov, *AsymptoticAnalysis.wl*, pinned repository source. Public entries and core request infrastructure.
- [2] Vladimir Reshetnikov, *InverseCertificates.wl*, pinned repository source. Exact enclosures, certificate attempt, and retry driver.
- [3] *Code review status*, pinned repository register. Consolidated findings and historical crosswalks.
- [4] *Wave 3 intake*, pinned repository register. Mechanism-level crosswalk and proposals.
- [5] *Wave 4 intake*, pinned repository register. Compatibility, validation, and adapter crosswalk.
- [6] *Wave 5 review index*, pinned repository. Latest review-package summaries and retired entries.
- [7] *MathicsCompatibility.wl*, pinned repository source. Scoped evaluator compatibility helpers.
- [8] *MathicsNumerical.wl*, pinned repository source. Numerical precision admission and logarithm retry helpers.
- [9] *SpecialFunctionIdentities.wl*, pinned repository source. Exact identities before asymptotic expansion.
- [10] *ParameterizedSpecialFunctions.wl*, pinned repository source. Fixed-parameter and Frobenius adapters.
- [11] *DirichletSpecialFunctions.wl*, pinned repository source. Zeta and Lerch defining-series expansions and bounds.
- [12] Wolfram Research, *Power*, Wolfram Language documentation, accessed September 10, 2026. <https://reference.wolfram.com/language/ref/Power.html>.
- [13] NIST Digital Library of Mathematical Functions, §10.39, *Relations to Other Functions*. <https://dlmf.nist.gov/10.39>.
- [14] NIST Digital Library of Mathematical Functions, §13.6, *Relations to Other Functions*. <https://dlmf.nist.gov/13.6>.
- [15] NIST Digital Library of Mathematical Functions, §8.7, *Series Expansions*. <https://dlmf.nist.gov/8.7>.
- [16] NIST Digital Library of Mathematical Functions, §25.14, *Lerch's Transcendent*. <https://dlmf.nist.gov/25.14>.